

basetable: A Complete Function Reference

Every exported function, explained progressively with real examples

Table of contents

1	Introduction	4
2	Core verbs	4
2.1	Subsetting rows and columns: <code>subset()</code>	5
2.2	Picking and dropping columns: <code>pick()</code> , <code>drop()</code>	5
2.3	Creating and modifying columns: <code>transform()</code> , <code>within()</code>	6
2.4	dplyr-style aliases: <code>filter()</code> , <code>select()</code> , <code>mutate()</code> , <code>transmute()</code> , <code>rename()</code> , <code>arrange()</code>	7
2.5	Sorting rows: <code>reorder()</code> , <code>arrange()</code>	8
3	Aggregation and grouping	9
3.1	Grouped summaries: <code>aggregate()</code>	9
3.2	Counting rows: <code>count()</code> , <code>propcount()</code>	9
3.3	Named summaries with <code>summarise()</code> / <code>summarize()</code> / <code>summaries()</code>	10
3.4	Splitting into groups: <code>split()</code> , <code>by_apply()</code> , <code>applyby()</code> , <code>combine()</code> , <code>recombine()</code>	11
3.5	First/last row per group: <code>firstby()</code> , <code>lastby()</code>	12
4	Joining tables	12
4.1	Standard joins: <code>merge()</code>	12
4.2	Semi- and anti-joins: <code>semimerge()</code> , <code>antimerge()</code>	13
4.3	Key-matching helpers: <code>matchedkeys()</code> , <code>unmatchedkeys()</code> , <code>joinrelationship()</code>	13
4.4	Cross joins: <code>crossmerge()</code>	14
4.5	Rolling and nearest-key joins: <code>rollingmerge()</code> , <code>nearestmerge()</code>	14
4.6	Interval overlap joins: <code>overlapmerge()</code>	15
4.7	Updating values from another table: <code>updatemerge()</code>	15
4.8	Conditional joins: <code>nonequimerge()</code> , <code>rangemerge()</code>	16

5	Set operations and table comparison	16
5.1	Row-level set operations: <code>unionrows()</code> , <code>intersectrows()</code> , <code>diffrows()</code>	16
5.2	Comparing two tables: <code>equalrows()</code> , <code>equaldata()</code> , <code>samechema()</code> , <code>compareschema()</code> , <code>changedcols()</code>	17
5.3	Comparing two snapshots of the same table: <code>addedrows()</code> , <code>removedrows()</code> , <code>changedrows()</code>	18
5.4	Row/column binding: <code>bind_rows()</code> , <code>bind_cols()</code> , <code>rbindfill()</code>	19
6	Reshaping	20
6.1	Long and wide formats: <code>toalong()</code> , <code>towide()</code>	20
6.2	Base R reshape, stack, and unstack: <code>reshape()</code> , <code>stack()</code> , <code>unstack()</code>	20
6.3	Splitting and combining text columns: <code>separate()</code> , <code>unite()</code>	21
6.4	Transposing: <code>transpose()</code>	21
7	Exploration and EDA	22
7.1	Shape and types: <code>dims()</code> , <code>types()</code> , <code>nrows()</code> , <code>ncols()</code>	22
7.2	Peeking at data: <code>headtail()</code> , <code>glimpse()</code>	22
7.3	Descriptive statistics: <code>describe()</code> , <code>profile()</code>	23
7.4	Frequency tables: <code>freq()</code>	23
7.5	Table 1-style summaries: <code>summarytab()</code>	24
7.6	Missingness: <code>missingness()</code>	24
7.7	Comparing two tables at a glance: <code>compare()</code>	25
8	Data cleaning and validation	26
8.1	Assertions that stop on failure: <code>assert*()</code>	26
8.2	Finding rows/values that fail a check: <code>invalidrows()</code> , <code>invalidvalues()</code> , <code>outofrange()</code>	27
8.3	Missing-data helpers: <code>missingrows()</code> , <code>keepmissing()</code> , <code>omitmissing()</code> , <code>missingindicator()</code>	28
8.4	Filling in missing combinations: <code>completegrid()</code> , <code>expandrows()</code>	29
8.5	Recoding values: <code>naif()</code> , <code>nato()</code> , <code>blanktona()</code> , <code>natoblank()</code> , <code>replacevalues()</code> , <code>replacewhere()</code> , <code>replacecols()</code>	29
8.6	Deduplication: <code>distinct()</code> , <code>removeduplicates()</code> , <code>duplicaterows()</code> , <code>duplicatekeys()</code> , <code>duplicatenames()</code>	30
8.7	Column-name hygiene: <code>cleannames()</code> , <code>repairnames()</code> , <code>renamewith()</code> , <code>commonnames()</code>	31
8.8	Filling in missing values within groups: <code>filldown()</code> , <code>fillup()</code> , <code>fillboth()</code> . .	32
9	Row and column utilities	33
9.1	Column metadata: <code>colnames()</code> , <code>rownames()</code> , <code>classes()</code> , <code>uniques()</code> , <code>cardinality()</code> , <code>constants()</code> , <code>emptycols()</code>	33
9.2	Finding blank rows: <code>emptyrows()</code>	34
9.3	Reordering columns: <code>move()</code> , <code>firstcols()</code> , <code>lastcols()</code>	34

9.4	Row subsets: <code>firstrows()</code> , <code>lastrows()</code> , <code>samplerows()</code> , <code>samplefrac()</code> , <code>slice()</code> , <code>reverse()</code>	35
9.5	Sorting rows: <code>orderrows()</code>	36
9.6	Column reordering: <code>relocate()</code>	36
9.7	Row position helpers: <code>rownumber()</code>	36
9.8	Row-wise reductions across columns: <code>rowmin()</code> , <code>rowmax()</code> , <code>rowany()</code> , <code>rowall()</code> , <code>rowcount()</code> , <code>rowfirst()</code> , <code>rowlast()</code> , <code>rowapply()</code>	36
9.9	Applying a function to selected columns: <code>applycols()</code> , <code>convertcols()</code>	37
10	String manipulation	38
10.1	Whitespace and case: <code>trim()</code> , <code>squish()</code> , <code>lower()</code> , <code>upper()</code> , <code>titlecase()</code> , <code>sentencecase()</code> , <code>textlen()</code>	38
10.2	Substrings and padding: <code>left()</code> , <code>right()</code> , <code>middle()</code> , <code>truncate()</code> , <code>padleft()</code> , <code>padright()</code> , <code>padcenter()</code>	39
10.3	Pattern matching: <code>contains()</code> , <code>matches()</code> , <code>startswith()</code> , <code>endswith()</code> , <code>countmatch()</code> , <code>locate()</code> , <code>locateall()</code>	39
10.4	Extraction: <code>extract()</code> , <code>extractall()</code> , <code>extractnum()</code> , <code>extractint()</code> , <code>extractbetween()</code>	40
10.5	Replace and remove: <code>replacetext()</code> , <code>removetext()</code> , <code>removeall()</code> , <code>replaceall()</code>	41
10.6	Split and join: <code>splittext()</code> , <code>splitfirst()</code> , <code>splitlast()</code> , <code>jointext()</code> , <code>collapsestext()</code>	41
10.7	Blank and encoding helpers: <code>isblank()</code> , <code>removeaccents()</code> , <code>normalizeunicode()</code> , <code>normalizeencoding()</code> , <code>transliterate()</code>	42
10.8	String distance and similarity: <code>textdist()</code> , <code>nearesttext()</code> , <code>similartext()</code> .	42
10.9	Classification predicates: <code>isalpha()</code> , <code>isalphanumeric()</code> , <code>isnumerictext()</code> , <code>isintegertext()</code> , <code>isemail()</code> , <code>isurl()</code>	43
10.10	Recoding and grouping values: <code>recode()</code> , <code>collapsevalues()</code> , <code>collapselevels()</code> , <code>lump()</code> , <code>reorderlevels()</code> , <code>expandlevels()</code>	43
11	Parsing text into typed values	44
12	Dates and times	45
12.1	Extracting components	45
12.2	Arithmetic and sequences: <code>adddays()</code> , <code>addweeks()</code> , <code>addmonths()</code> , <code>addyears()</code> , <code>datediff()</code> , <code>dateseq()</code> , <code>between dates()</code>	46
12.3	Rounding dates: <code>floordate()</code> , <code>ceilingdate()</code> , <code>rounddate()</code>	47
13	Numeric transforms and rolling windows	47
13.1	Rescaling and standardizing: <code>rescale()</code> , <code>standardize()</code> , <code>center()</code> , <code>winsorize()</code>	47
13.2	Binning: <code>quantilegroup()</code>	48

13.3 Period-over-period change and rank: <code>percentchange()</code> , <code>percentrank()</code> , <code>denserank()</code>	48
13.4 Cumulative helpers: <code>cumcount()</code> , <code>cumedist()</code> , <code>cummean()</code>	48
13.5 Lag/lead and differencing: <code>difference()</code> , <code>lagvalue()</code> , <code>leadvalue()</code>	49
13.6 Rolling window functions: <code>rollmean()</code> , <code>rollsum()</code> , <code>rollmin()</code> , <code>rollmax()</code> , <code>rollmedian()</code> , <code>rollsd()</code> , <code>rollvar()</code> , <code>rollprod()</code> , <code>rollapply()</code>	49
14 Functional helpers	50
15 Closing notes	51

1 Introduction

`basetable` is a compact R package for tabular data manipulation and exploratory data analysis. It has three design goals:

- **Base-style naming and semantics.** Functions read like `subset()`, `transform()`, `aggregate()`, `merge()`, and `split()` — the base R verbs most R users already know — rather than inventing a new grammar.
- **`data.table` as the execution backend.** Every operation that touches real data routes through `data.table`, so the package is fast without asking you to learn `data.table`'s `[i, j, by]` syntax.
- **Explicit, standard-evaluation interfaces.** Column names are passed as character strings, not bare symbols captured by non-standard evaluation (with a few clearly-marked exceptions, like `filter()`'s and `transform()`'s expression arguments, which mirror base R's own `subset()/transform()` behavior).

This document walks through **every function `basetable` exports**, grouped into sections that build on each other: start with the core verbs, move through aggregation, joining, reshaping, and exploration, then into the lower-level data-cleaning, string, date, and numeric utility functions. Read it top to bottom to learn the package, or jump to a section as a reference.

Every example below is real, runnable code — the output shown is the actual output `basetable` produces.

2 Core verbs

These are the everyday functions: subsetting rows, picking columns, and creating new columns. If you know `mtcars`, you know enough to follow along.

2.1 Subsetting rows and columns: subset()

subset() mirrors base::subset() exactly: a logical condition for rows, and an optional select for columns.

```
subset(mtcars, cyl == 6, select = c("mpg", "hp", "wt"))
```

	mpg	hp	wt
	<num>	<num>	<num>
1:	21.0	110	2.620
2:	21.0	110	2.875
3:	21.4	110	3.215
4:	18.1	105	3.460
5:	19.2	123	3.440
6:	17.8	123	3.440
7:	19.7	175	2.770

2.2 Picking and dropping columns: pick(), drop()

pick() keeps only the named columns; drop() removes them.

```
pick(mtcars, c("mpg", "hp"))
```

```
# A tibble: 32 x 2
  mpg    hp
<dbl> <dbl>
1  21    110
2  21    110
3 22.8    93
4 21.4    110
5 18.7    175
6 18.1    105
7 14.3    245
8 24.4     62
9 22.8     95
10 19.2    123
# i 22 more rows
```

```
drop(mtcars, c("vs", "am"))
```

```
# A tibble: 32 x 9
  mpg   cyl  disp    hp  drat    wt   qsec    gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  21       6  160   110  3.9   2.62  16.5     4     4
2  21       6  160   110  3.9   2.88  17.0     4     4
3  22.8     4  108    93  3.85  2.32  18.6     4     1
4  21.4     6  258   110  3.08  3.22  19.4     3     1
5  18.7     8  360   175  3.15  3.44  17.0     3     2
6  18.1     6  225   105  2.76  3.46  20.2     3     1
7  14.3     8  360   245  3.21  3.57  15.8     3     4
8  24.4     4  147.    62  3.69  3.19  20       4     2
9  22.8     4  141.    95  3.92  3.15  22.9     4     2
10 19.2     6  168.   123  3.92  3.44  18.3     4     4
# i 22 more rows
```

2.3 Creating and modifying columns: transform(), within()

`transform()` adds or replaces columns using named expressions, exactly like `base::transform()`. Later expressions in the same call can refer to columns created earlier in that call.

```
transform(mtcars, power = hp / wt, power_sq = power^2)
```

```
# A tibble: 32 x 13
  mpg   cyl  disp    hp  drat    wt   qsec    vs    am  gear  carb power power_sq
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  21       6  160   110  3.9   2.62  16.5     0     1     4     4  42.0  1763.
2  21       6  160   110  3.9   2.88  17.0     0     1     4     4  38.3  1464.
3  22.8     4  108    93  3.85  2.32  18.6     1     1     4     1  40.1  1607.
4  21.4     6  258   110  3.08  3.22  19.4     1     0     3     1  34.2  1171.
5  18.7     8  360   175  3.15  3.44  17.0     0     0     3     2  50.9  2588.
6  18.1     6  225   105  2.76  3.46  20.2     1     0     3     1  30.3   921.
7  14.3     8  360   245  3.21  3.57  15.8     0     0     3     4  68.6  4710.
8  24.4     4  147.    62  3.69  3.19  20       1     0     4     2  19.4   378.
9  22.8     4  141.    95  3.92  3.15  22.9     1     0     4     2  30.2   910.
10 19.2     6  168.   123  3.92  3.44  18.3     1     0     4     4  35.8  1278.
# i 22 more rows
```

`within()` instead evaluates a whole block of code against the data's columns and returns whatever new/changed variables come out — this is `basetable`'s version of `base::within()`.

```
within(mtcars, {
  power <- hp / wt
  heavy <- wt > 3.5
})
```

```
# A tibble: 32 x 13
```

	heavy	power	carb	gear	am	vs	qsec	wt	drat	hp	disp	cyl	mpg
	<lgl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	FALSE	42.0	4	4	1	0	16.5	2.62	3.9	110	160	6	21
2	FALSE	38.3	4	4	1	0	17.0	2.88	3.9	110	160	6	21
3	FALSE	40.1	1	4	1	1	18.6	2.32	3.85	93	108	4	22.8
4	FALSE	34.2	1	3	0	1	19.4	3.22	3.08	110	258	6	21.4
5	FALSE	50.9	2	3	0	0	17.0	3.44	3.15	175	360	8	18.7
6	FALSE	30.3	1	3	0	1	20.2	3.46	2.76	105	225	6	18.1
7	TRUE	68.6	4	3	0	0	15.8	3.57	3.21	245	360	8	14.3
8	FALSE	19.4	2	4	0	1	20	3.19	3.69	62	147.	4	24.4
9	FALSE	30.2	2	4	0	1	22.9	3.15	3.92	95	141.	4	22.8
10	FALSE	35.8	4	4	0	1	18.3	3.44	3.92	123	168.	6	19.2

```
# i 22 more rows
```

2.4 dplyr-style aliases: filter(), select(), mutate(), transmute(), rename(), arrange()

For readers coming from dplyr, `basetable` provides familiar verb names. These are thin wrappers: `filter()` is `subset()`'s row logic, `select()` is `pick()`, `mutate()/transmute()` are `transform()`, and `arrange()` is `reorder()` (below).

```
mtcars |>
  filter(cyl == 6, mpg > 18) |>
  select(c("mpg", "cyl", "hp")) |>
  mutate(ratio = hp / mpg) |>
  arrange("ratio", decreasing = TRUE)
```

```
# A tibble: 6 x 4
```

	mpg	cyl	hp	ratio
	<dbl>	<dbl>	<dbl>	<dbl>
1	19.7	6	175	8.88
2	19.2	6	123	6.41
3	18.1	6	105	5.80
4	21	6	110	5.24

```
5  21      6   110  5.24
6  21.4    6   110  5.14
```

```
transmute(mtcars, ratio = hp / mpg)
```

```
# A tibble: 32 x 1
  ratio
  <dbl>
1  5.24
2  5.24
3  4.08
4  5.14
5  9.36
6  5.80
7 17.1
8  2.54
9  4.17
10 6.41
# i 22 more rows
```

```
rename(mtcars, horsepower = hp) |> pick(c("horsepower", "mpg"))
```

```
# A tibble: 32 x 2
  horsepower mpg
  <dbl> <dbl>
1      110  21
2      110  21
3       93 22.8
4      110 21.4
5      175 18.7
6      105 18.1
7      245 14.3
8       62 24.4
9       95 22.8
10     123 19.2
# i 22 more rows
```

2.5 Sorting rows: reorder(), arrange()

`reorder()` sorts a table by one or more columns, ascending by default. `arrange()` is an identical alias.

```
reorder(mtcars, by = c("cyl", "mpg"), decreasing = c(FALSE, TRUE)) |> headtail(3)
```

```
# A tibble: 6 x 11
  mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  33.9     4  71.1    65  4.22  1.84  19.9     1   1     4     1
2  32.4     4  78.7    66  4.08  2.2   19.5     1   1     4     1
3  30.4     4  75.7    52  4.93  1.62  18.5     1   1     4     2
4  13.3     8 350     245  3.73  3.84  15.4     0   0     3     4
5  10.4     8 472     205  2.93  5.25  18.0     0   0     3     4
6  10.4     8 460     215   3     5.42  17.8     0   0     3     4
```

3 Aggregation and grouping

3.1 Grouped summaries: `aggregate()`

`aggregate()` computes one summary function over one or more value columns, per group — the `data.table`-backed analogue of `stats::aggregate()`. Note it returns a `data.table` directly (not a tibble), since it's meant to be fast and composable with further `data.table` operations.

```
aggregate(mtcars, by = "cyl", value = c("mpg", "hp"), fun = mean)
```

```
      cyl   mpg    hp
<num> <num> <num>
1:     4 26.66 82.64
2:     6 19.74 122.29
3:     8 15.10 209.21
```

3.2 Counting rows: `count()`, `propcount()`

`count()` counts rows per group; `propcount()` adds proportions, optionally computed within a margin of columns rather than over the whole table.

```
count(iris, by = "Species")
```

```
# A tibble: 3 x 2
  Species      n
  <fct>    <int>
1 setosa     50
2 versicolor 50
3 virginica  50
```

```
propcount(mtcars, by = c("cyl", "am"), margin = "cyl")
```

```
# A tibble: 6 x 4
  cyl    am      n prop
  <dbl> <dbl> <int> <dbl>
1     6     1      3 0.429
2     4     1      8 0.727
3     6     0      4 0.571
4     8     0     12 0.857
5     4     0      3 0.273
6     8     1      2 0.143
```

3.3 Named summaries with summarise() / summarize() / summaries()

`summarise()` (and its `summarize()` alias) take named summary expressions directly, dplyr-style, with `by` as a named argument:

```
summarise(mtcars, mean_mpg = mean(mpg), n = length(mpg), by = "cyl")
```

```
# A tibble: 3 x 3
  cyl mean_mpg      n
  <dbl>   <dbl> <int>
1     4    26.7    11
2     6    19.7     7
3     8    15.1    14
```

`summaries()` is the same idea with `by` as the second positional argument — useful when you're calling it programmatically:

```
summaries(mtcars, "cyl", mean_hp = mean(hp), sd_hp = sd(hp))
```

```
# A tibble: 3 x 3
  cyl mean_hp sd_hp
<dbl> <dbl> <dbl>
1     4   82.6  20.9
2     6  122.  24.3
3     8  209.  51.0
```

3.4 Splitting into groups: `split()`, `by_apply()`, `applyby()`, `combine()`, `recombine()`

`split()` breaks a table into a named list of tibbles, one per group:

```
pieces <- split(iris, by = "Species")
names(pieces)
```

```
[1] "setosa"      "versicolor" "virginica"
```

```
pieces$setosa |> headtail(2)
```

```
# A tibble: 4 x 4
  Sepal.Length Sepal.Width Petal.Length Petal.Width
      <dbl>      <dbl>      <dbl>      <dbl>
1       5.1       3.5       1.4       0.2
2       4.9       3       1.4       0.2
3       5.3       3.7       1.5       0.2
4        5       3.3       1.4       0.2
```

`by_apply()` (and its close relative `applyby()`) apply a function to each group and, if `bind = TRUE`, stitch the results back into one table via `combine()` (whose alias is `recombine()`):

```
by_apply(mtcars, by = "cyl", fun = function(d) data.frame(mean_mpg = mean(d$mpg)), bind = TRUE)
```

```
# A tibble: 3 x 2
  cyl_group mean_mpg
  <chr>      <dbl>
1 4         26.7
2 6         19.7
3 8         15.1
```

3.5 First/last row per group: firstby(), lastby()

```
df <- data.frame(id = c(1, 1, 2, 2, 2), visit = c(1, 2, 1, 2, 3), value = c(10, 12, 5, 6, 7))
firstby(df, by = "id")
```

```
# A tibble: 2 x 3
  id visit value
<dbl> <dbl> <dbl>
1     1     1    10
2     2     1     5
```

```
lastby(df, by = "id")
```

```
# A tibble: 2 x 3
  id visit value
<dbl> <dbl> <dbl>
1     1     2    12
2     2     3     7
```

4 Joining tables

basetable provides the full family of table joins, all `data.table`-backed.

4.1 Standard joins: merge()

`merge()` mirrors `base::merge()`'s interface exactly (`by`, `all`, `all.x`, `all.y`, `suffixes`) while running on `data.table` internally.

```
orders <- data.frame(id = c(1, 2, 3), customer = c("a", "b", "c"))
payments <- data.frame(id = c(2, 3, 4), amount = c(50, 75, 20))

merge(orders, payments, by = "id")
```

```
   id customer amount
<num>   <char>   <num>
1:    2       b     50
2:    3       c     75
```

```
merge(orders, payments, by = "id", all.x = TRUE)
```

	id	customer	amount
	<num>	<char>	<num>
1:	1	a	NA
2:	2	b	50
3:	3	c	75

4.2 Semi- and anti-joins: `semimerge()`, `antimerge()`

`semimerge()` keeps rows of `x` whose key exists in `y` (like `merge()` but without adding `y`'s columns); `antimerge()` keeps rows of `x` whose key does **not** exist in `y`.

```
semimerge(orders, payments, by = "id")
```

	id	customer
	<num>	<char>
1:	2	b
2:	3	c

```
antimerge(orders, payments, by = "id")
```

	id	customer
	<num>	<char>
1:	1	a

4.3 Key-matching helpers: `matchedkeys()`, `unmatchedkeys()`, `joinrelationship()`

`matchedkeys()`/`unmatchedkeys()` return `x`'s distinct rows whose key is (or isn't) present in `y`. `joinrelationship()` classifies the cardinality of a join key between two tables.

```
matchedkeys(orders, payments, by = "id")
```

	id	customer
	<num>	<char>
1:	2	b
2:	3	c

```
unmatchedkeys(orders, payments, by = "id")
```

```
      id customer  
    <num>   <char>  
1:      1         a
```

```
joinrelationship(orders, payments, by = "id")
```

```
[1] "one-to-one"
```

4.4 Cross joins: crossmerge()

crossmerge() returns the full Cartesian product of two tables.

```
crossmerge(data.frame(size = c("S", "M")), data.frame(color = c("red", "blue")))
```

```
      size color  
    <char> <char>  
1:       S   red  
2:       S  blue  
3:       M   red  
4:       M  blue
```

4.5 Rolling and nearest-key joins: rollingmerge(), nearestmerge()

rollingmerge() matches each row of x to the nearest row of y **at or before** it (direction = "backward"), at or after it ("forward"), or whichever is numerically closest ("nearest") — the classic “as-of” join used for time series and event logs. nearestmerge() is a convenience wrapper for direction = "nearest".

```
trades <- data.frame(id = 1L, time = c(5, 10, 15))  
quotes <- data.frame(id = 1L, time = c(3, 8, 14), price = c(100, 101, 99))  
  
rollingmerge(trades, quotes, by = c("id", "time"), direction = "backward")
```

```
      id time price  
    <int> <num> <num>  
1:      1     5  100  
2:      1    10  101  
3:      1    15   99
```

```
nearestmerge(trades, quotes, by = c("id", "time"))
```

	id	time	price
	<int>	<num>	<num>
1:	1	5	100
2:	1	10	101
3:	1	15	99

4.6 Interval overlap joins: overlapmerge()

`overlapmerge()` matches rows whose `[startx, endx]` interval overlaps a `[starty, endy]` interval in `y` (optionally within an exact `by` key) — for example, matching events to the time windows they fall inside.

```
events <- data.frame(id = 1L, start = c(1, 10), end = c(5, 15))
windows <- data.frame(id = 1L, start = c(0, 8), end = c(6, 20), label = c("early", "late"))
overlapmerge(events, windows, startx = "start", endx = "end", starty = "start", endy = "end")
```

	id	start	end	label	start	end
	<int>	<num>	<num>	<char>	<num>	<num>
1:	1	0	6	early	1	5
2:	1	8	20	late	10	15

4.7 Updating values from another table: updatemerge()

`updatemerge()` overwrites `x`'s values with `y`'s wherever the join key matches (an in-place “upsert” of column values, not a column-adding merge).

```
current <- data.frame(id = c(1, 2, 3), status = c("pending", "pending", "pending"))
updates <- data.frame(id = c(2, 3), status = c("shipped", "cancelled"))
updatemerge(current, updates, by = "id")
```

	id	status
	<num>	<char>
1:	1	pending
2:	2	shipped
3:	3	cancelled

4.8 Conditional joins: `nonequimerge()`, `rangemerge()`

`nonequimerge()` supports genuine inequality conditions in `by`, written as "`xcol<op>ycol`" (the same convention `data.table` itself uses for `on=`), alongside any plain exact-match columns:

```
events <- data.frame(id = 1, date = as.Date("2024-01-15"))
periods <- data.frame(
  id = 1,
  start_date = as.Date(c("2024-01-01", "2024-02-01")),
  end_date = as.Date(c("2024-01-31", "2024-02-28")),
  period = c("Jan", "Feb")
)
nonequimerge(events, periods, by = c("id", "date>=start_date", "date<=end_date"))
```

	id	date	date.1	period
	<num>	<Date>	<Date>	<char>
1:	1	2024-01-01	2024-01-31	Jan

`rangemerge()` keeps every row of `x` (a table of ranges), attaching the row(s) of `y` whose `value` column falls inside that range, matched within `by`. An `x` row with no `y` row in range still appears once, with `NA` for `y`'s columns.

```
ranges <- data.frame(id = c(1, 1, 2), lower = c(0, 0, 0), upper = c(10, 10, 20))
points <- data.frame(id = c(1, 1, 2), val = c(5, 15, 5), label = c("a", "b", "c"))
rangemerge(ranges, points, by = "id", lower = "lower", upper = "upper", value = "val")
```

	id	val	label	lower	upper
	<num>	<num>	<char>	<num>	<num>
1:	1	5	a	0	10
2:	1	5	a	0	10
3:	2	5	c	0	20

5 Set operations and table comparison

5.1 Row-level set operations: `unionrows()`, `intersectrows()`, `diffrows()`

```
a <- data.frame(id = c(1, 2, 3))
b <- data.frame(id = c(2, 3, 4))

unionrows(a, b)
```

```
# A tibble: 4 x 1
  id
<dbl>
1    1
2    2
3    3
4    4
```

```
intersectrows(a, b, by = "id")
```

```
# A tibble: 2 x 1
  id
<dbl>
1    2
2    3
```

```
diffrows(a, b, by = "id")
```

```
# A tibble: 1 x 1
  id
<dbl>
1    1
```

5.2 Comparing two tables: equalrows(), equaldata(), sameschema(), compareschema(), changedcols()

equalrows() compares two tables' rows by key (order-insensitive, duplicate-sensitive). equaldata() compares full table content. sameschema() and compareschema() (aliased for this purpose by changedcols()) compare column names and types.

```
equalrows(a, data.frame(id = c(3, 2, 1)), by = "id")
```

```
[1] TRUE
```

```
equaldata(a, a[nrow(a):1, , drop = FALSE], ignoreorder = TRUE)
```

```
[1] TRUE
```

```
sameschema(mtcars, mtcars)
```

```
[1] TRUE
```

```
compareschema(mtcars, iris)
```

```
# A tibble: 16 x 5
  column      in_x in_y type_x type_y
  <chr>      <lgl> <lgl> <chr>  <chr>
1 mpg        TRUE FALSE double <NA>
2 cyl        TRUE FALSE double <NA>
3 disp       TRUE FALSE double <NA>
4 hp         TRUE FALSE double <NA>
5 drat       TRUE FALSE double <NA>
6 wt         TRUE FALSE double <NA>
7 qsec       TRUE FALSE double <NA>
8 vs         TRUE FALSE double <NA>
9 am         TRUE FALSE double <NA>
10 gear      TRUE FALSE double <NA>
11 carb      TRUE FALSE double <NA>
12 Sepal.Length FALSE TRUE  <NA>  double
13 Sepal.Width  FALSE TRUE  <NA>  double
14 Petal.Length FALSE TRUE  <NA>  double
15 Petal.Width  FALSE TRUE  <NA>  double
16 Species      FALSE TRUE  <NA>  integer
```

5.3 Comparing two snapshots of the same table: `addedrows()`, `removedrows()`, `changedrows()`

```
old <- data.frame(id = c(1, 2, 3), status = c("a", "b", "c"))
new <- data.frame(id = c(2, 3, 4), status = c("b", "c2", "d"))

addedrows(old, new, by = "id")
```

```
# A tibble: 1 x 2
  id status
  <dbl> <chr>
1     4 d
```

```
removedrows(old, new, by = "id")
```

```
# A tibble: 1 x 2
  id status
<dbl> <chr>
1     1 a
```

```
changedrows(old, new, by = "id")
```

```
# A tibble: 2 x 3
  id status.old status.new
<dbl> <chr>      <chr>
1     2 b        b
2     3 c        c2
```

5.4 Row/column binding: bind_rows(), bind_cols(), rbindfill()

`bind_rows()` stacks tables, filling missing columns with NA. `rbindfill()` is its lower-level building block. `bind_cols()` binds tables side by side.

```
bind_rows(data.frame(x = 1), data.frame(x = 2, y = 3))
```

```
# A tibble: 2 x 2
  x     y
<dbl> <dbl>
1     1 NA
2     2 3
```

```
bind_cols(data.frame(a = 1:2), data.frame(b = 3:4))
```

```
# A tibble: 2 x 2
  a     b
<int> <int>
1     1 3
2     2 4
```

6 Reshaping

6.1 Long and wide formats: `to_long()`, `to_wide()`

```
wide <- data.frame(id = 1:2, jan = c(10, 20), feb = c(11, 22))
long <- to_long(wide, cols = c("jan", "feb"), names = "month", values = "sales")
long
```

```
# A tibble: 4 x 3
      id month sales
  <int> <fct> <dbl>
1     1  jan     10
2     2  jan     20
3     1  feb     11
4     2  feb     22
```

```
to_wide(long, names = "month", values = "sales", idcols = "id", fun = sum)
```

```
# A tibble: 2 x 3
      id  jan  feb
  <int> <dbl> <dbl>
1     1    10    11
2     2    20    22
```

6.2 Base R reshape, stack, and unstack: `reshape()`, `stack()`, `unstack()`

`reshape()`, `stack()`, and `unstack()` are `base`-native wrappers around `stats::reshape()`, `utils::stack()`, and `utils::unstack()`, kept for base-R-compatibility rather than reimplemented on `data.table`.

```
wide2 <- data.frame(id = 1:2, t1 = c(10, 20), t2 = c(11, 22))
reshape(wide2, varying = c("t1", "t2"), v.names = "value", timevar = "time", idvar = "id", d
```

```
      id time value
1.1  1     1     10
2.1  2     1     20
1.2  1     2     11
2.2  2     2     22
```

```
stack(data.frame(a = 1:2, b = 3:4))
```

```
  values ind
1      1  a
2      2  a
3      3  b
4      4  b
```

6.3 Splitting and combining text columns: `separate()`, `unite()`

```
codes <- data.frame(code = c("A-1-red", "B-2-blue"))
parts <- separate(codes, column = "code", into = c("letter", "num", "color"), sep = "-")
parts
```

```
# A tibble: 2 x 3
  letter num  color
  <chr> <chr> <chr>
1 A      1    red
2 B      2   blue
```

```
unite(parts, column = "code", cols = c("letter", "num", "color"), sep = "_")
```

```
# A tibble: 2 x 1
  code
  <chr>
1 A_1_red
2 B_2_blue
```

6.4 Transposing: `transpose()`

```
transpose(data.frame(a = 1:2, b = 3:4))
```

```
  V1 V2
1  1  2
2  3  4
```

7 Exploration and EDA

7.1 Shape and types: `dims()`, `types()`, `nrows()`, `ncols()`

```
dims(iris)
```

```
# A tibble: 1 x 2
  rows cols
  <int> <int>
1   150     5
```

```
types(iris)
```

```
# A tibble: 5 x 3
  column      class  typeof
  <chr>      <chr>  <chr>
1 Sepal.Length numeric double
2 Sepal.Width  numeric double
3 Petal.Length numeric double
4 Petal.Width  numeric double
5 Species      factor  integer
```

7.2 Peeking at data: `headtail()`, `glimpse()`

```
headtail(iris, 2)
```

```
# A tibble: 4 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>      <dbl>      <dbl>      <dbl> <fct>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         6.2         3.4         5.4         2.3 virginica
4         5.9         3         5.1         1.8 virginica
```

```
glimpse(iris)
```

```

Rows: 150 Columns: 5
$ Sepal.Length <numeric> 5.1, 4.9, 4.7
$ Sepal.Width <numeric> 3.5, 3, 3.2
$ Petal.Length <numeric> 1.4, 1.4, 1.3
$ Petal.Width <numeric> 0.2, 0.2, 0.2
$ Species <factor> setosa, setosa, setosa

```

7.3 Descriptive statistics: describe(), profile()

describe() gives a per-column summary (mean, sd, quantiles for numeric columns; top values for others). profile() is an alias.

```
describe(iris)
```

```

# A tibble: 5 x 14
  column      class      n missing missing_prop distinct  mean    sd   min  q25 median  q75
  <chr>      <chr> <int>   <int>         <dbl>   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Sepal.L~ nume~   150     0           0      35  5.84  0.828  4.3   5.1   5.8   6.4
2 Sepal.W~ nume~   150     0           0      23  3.06  0.436  2    2.8   3    3.3
3 Petal.L~ nume~   150     0           0      43  3.76  1.77   1    1.6  4.35   5.1
4 Petal.W~ nume~   150     0           0      22  1.20  0.762  0.1   0.3   1.3   1.8
5 Species  fact~   150     0           0       3 NA    NA     NA    NA    NA    NA
# i 2 more variables: max <dbl>, top <chr>

```

7.4 Frequency tables: freq()

```
freq(iris, column = "Species")
```

```

# A tibble: 3 x 2
  Species      n
  <fct>    <int>
1 setosa    50
2 versicolor 50
3 virginica  50

```

```
freq(mtcars, column = "cyl", by = "am", prop = TRUE)
```

```
# A tibble: 6 x 4
      am    cyl      n prop
  <dbl> <dbl> <int> <dbl>
1     0     8    12 0.632
2     1     4     8 0.615
3     0     6     4 0.211
4     1     6     3 0.231
5     0     4     3 0.158
6     1     8     2 0.154
```

7.5 Table 1-style summaries: summarytab()

```
dat <- transform(mtcars, am = factor(am, labels = c("Automatic", "Manual")))
summarytab(dat, vars = c("mpg", "cyl"), by = "am", p_value = TRUE)
```

```
# A tibble: 2 x 6
  variable level      Automatic Manual      Overall p_value
  <chr>      <chr>      <chr>      <chr>      <chr>      <chr>
1 mpg      Mean (SD) 17.1 (3.8) 24.4 (6.2) 20.1 (6.0) 0.00137
2 cyl      Mean (SD)  6.9 (1.5)  5.1 (1.6)  6.2 (1.8) 0.00246
```

7.6 Missingness: missingness()

```
with_na <- transform(iris, Sepal.Length = ifelse(Sepal.Length > 7, NA, Sepal.Length))
missingness(with_na)
```

```
# A tibble: 5 x 4
  column      missing missing_prop complete
  <chr>      <dbl>      <dbl>      <dbl>
1 Sepal.Length    12         0.08    138
2 Sepal.Width      0          0     150
3 Petal.Length      0          0     150
4 Petal.Width      0          0     150
5 Species          0          0     150
```

7.7 Comparing two tables at a glance: compare()

```
compare(iris, with_na, by = "Species")
```

\$dims

```
# A tibble: 2 x 3
  object rows cols
  <chr>  <int> <int>
1 x         150     5
2 y         150     5
```

\$names

```
# A tibble: 5 x 3
  column      in_x in_y
  <chr>      <lgl> <lgl>
1 Sepal.Length TRUE  TRUE
2 Sepal.Width  TRUE  TRUE
3 Petal.Length TRUE  TRUE
4 Petal.Width  TRUE  TRUE
5 Species      TRUE  TRUE
```

\$types

```
# A tibble: 5 x 5
  column      class.x typeof.x class.y typeof.y
  <chr>      <chr>  <chr>   <chr>  <chr>
1 Sepal.Length numeric double  numeric double
2 Sepal.Width  numeric double  numeric double
3 Petal.Length numeric double  numeric double
4 Petal.Width  numeric double  numeric double
5 Species      factor  integer factor  integer
```

\$missing

```
# A tibble: 5 x 7
  column      missing.x missing_prop.x complete.x missing.y missing_prop.y complete.y
  <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1 Sepal.Length      0          0        150        12        0.08        138
2 Sepal.Width        0          0        150         0         0         150
3 Petal.Length        0          0        150         0         0         150
4 Petal.Width        0          0        150         0         0         150
5 Species            0          0        150         0         0         150
```

```
$key_overlap
# A tibble: 1 x 3
  x_unique y_unique common
  <int>    <int>    <int>
1       3       3       3
```

8 Data cleaning and validation

8.1 Assertions that stop on failure: `assert*()`

Each `assert*()` function returns its input invisibly if the check passes, and throws an informative error otherwise — useful as a pipeline guard.

```
mtcars |>
  assertnames(c("mpg", "cyl")) |>
  assertrows(mpg > 0) |>
  assertrange("mpg", lower = 0, upper = 60) |>
  asserttype("cyl", "numeric") |>
  assertcomplete() |>
  headtail(2)
```

```
# A tibble: 4 x 11
  mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  21     6   160   110   3.9   2.62  16.5     0     1     4     4
2  21     6   160   110   3.9   2.88  17.0     0     1     4     4
3  15     8   301   335   3.54  3.57  14.6     0     1     5     8
4 21.4     4   121   109   4.11  2.78  18.6     1     1     4     2
```

```
tryCatch(assertkey(mtcars, "cyl"), error = function(e) conditionMessage(e))
```

```
[1] "`by` does not identify unique rows."
```

```
tryCatch(assertunique(mtcars, "cyl"), error = function(e) conditionMessage(e))
```

```
[1] "Values are not unique."
```

```
tryCatch(assertvalues(mtcars, "am", allowed = c(0, 1)), error = function(e) "passes")
```

`assertcols()`/`assertkey()` are convenience aliases for the lower-level `assert_cols()`/`assert_key()`, which do the same checks and are what you'd call from code that doesn't need the `assert*` naming convention:

```
assert_cols(mtcars, c("mpg", "cyl")) |> invisible()
tryCatch(assert_key(mtcars, "cyl"), error = function(e) conditionMessage(e))
```

```
[1] "`by` does not identify unique rows."
```

8.2 Finding rows/values that fail a check: `invalidrows()`, `invalidvalues()`, `outofrange()`

These are the non-throwing counterparts of the `assert*()` functions above — instead of stopping, they return the offending rows.

```
invalidrows(mtcars, mpg > 15)
```

```
# A tibble: 6 x 11
  mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  14.3     8   360   245  3.21  3.57  15.8     0     0     3     4
2  10.4     8   472   205  2.93  5.25  18.0     0     0     3     4
3  10.4     8   460   215    3   5.42  17.8     0     0     3     4
4  14.7     8   440   230  3.23  5.34  17.4     0     0     3     4
5  13.3     8   350   245  3.73  3.84  15.4     0     0     3     4
6   15     8   301   335  3.54  3.57  14.6     0     1     5     8
```

```
outofrange(mtcars, "mpg", lower = 15, upper = 30)
```

```
# A tibble: 9 x 11
  mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  14.3     8   360   245  3.21  3.57  15.8     0     0     3     4
2  10.4     8   472   205  2.93  5.25  18.0     0     0     3     4
3  10.4     8   460   215    3   5.42  17.8     0     0     3     4
4  14.7     8   440   230  3.23  5.34  17.4     0     0     3     4
5  32.4     4   78.7    66  4.08  2.2   19.5     1     1     4     1
```

6	30.4	4	75.7	52	4.93	1.62	18.5	1	1	4	2
7	33.9	4	71.1	65	4.22	1.84	19.9	1	1	4	1
8	13.3	8	350	245	3.73	3.84	15.4	0	0	3	4
9	30.4	4	95.1	113	3.77	1.51	16.9	1	1	5	2

8.3 Missing-data helpers: `missingrows()`, `keepmissing()`, `omitmissing()`, `missingindicator()`

```
d <- data.frame(a = c(1, NA, 3), b = c(NA, NA, 3))
missingrows(d, mode = "any")
```

```
# A tibble: 2 x 2
      a      b
  <dbl> <dbl>
1     1    NA
2    NA    NA
```

```
keepmissing(d)
```

```
# A tibble: 2 x 2
      a      b
  <dbl> <dbl>
1     1    NA
2    NA    NA
```

```
omitmissing(d, mode = "any")
```

```
# A tibble: 1 x 2
      a      b
  <dbl> <dbl>
1     3     3
```

```
missingindicator(d)
```

```
# A tibble: 3 x 4
      a      b missing_a missing_b
  <dbl> <dbl> <lgl>      <lgl>
1     1    NA FALSE      TRUE
2    NA    NA TRUE       TRUE
3     3     3 FALSE      FALSE
```

8.4 Filling in missing combinations: `completegrid()`, `expandrows()`

`completegrid()` fills in every combination of the named columns that's missing from the data (useful before time-series analysis). `expandrows()` repeats each row a given number of times.

```
sparse <- data.frame(site = c("A", "A", "B"), year = c(2020, 2021, 2020), value = c(1, 2, 3))
completegrid(sparse, cols = c("site", "year"), fill = list(value = 0))
```

```
# A tibble: 4 x 3
  site   year value
  <chr> <dbl> <dbl>
1 A     2020     1
2 A     2021     2
3 B     2020     3
4 B     2021     0
```

```
expandrows(data.frame(x = c(1, 2)), times = c(2, 1))
```

```
# A tibble: 3 x 1
  x
  <dbl>
1     1
2     1
3     2
```

8.5 Recoding values: `naif()`, `nato()`, `blanktona()`, `natoblank()`, `replacevalues()`, `replacewhere()`, `replacecols()`

```
naif(c(1, 99, 2), 99)
```

```
[1] 1 NA 2
```

```
nato(c(1, NA, 2), 0)
```

```
[1] 1 0 2
```

```
blanktona(c("a", "", "b"))
```

```
[1] "a" NA  "b"
```

```
replacevalues(c("a", "b", "c"), old = c("a", "b"), new = c("A", "B"))
```

```
[1] "A" "B" "c"
```

```
replacewhere(mtcars, cyl == 4, cols = "mpg", value = NA) |> headtail(2)
```

```
# A tibble: 4 x 11
  mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    21     6   160   110   3.9   2.62  16.5     0     1     4     4
2    21     6   160   110   3.9   2.88  17.0     0     1     4     4
3    15     8   301   335   3.54  3.57  14.6     0     1     5     8
4    NA     4   121   109   4.11  2.78  18.6     1     1     4     2
```

8.6 Deduplication: distinct(), removeduplicates(), duplicaterows(), duplicatekeys(), duplicatenames()

```
dup_df <- data.frame(id = c(1, 1, 2), v = c("a", "b", "c"))
distinct(dup_df, cols = "id")
```

```
# A tibble: 2 x 1
  id
<dbl>
1    1
2    2
```

```
removeduplicates(dup_df, by = "id", keep = "first")
```

```
# A tibble: 2 x 2
  id v
<dbl> <chr>
1    1 a
2    2 c
```

```
duplicate_rows(data.frame(x = c(1, 1, 2)))
```

```
  x  
1 1  
2 1
```

```
duplicate_keys(dup_df, "id")
```

```
# A tibble: 1 x 2  
  id      N  
<dbl> <int>  
1     1     2
```

`duplicate_keys()` is a convenience alias for the lower-level `duplicated_keys()`:

```
duplicated_keys(dup_df, "id")
```

```
# A tibble: 1 x 2  
  id      N  
<dbl> <int>  
1     1     2
```

8.7 Column-name hygiene: `cleannames()`, `repairnames()`, `renamewith()`, `commonnames()`

```
messy <- data.frame(`First Name` = 1, `2nd_col` = 2, check.names = FALSE)  
cleannames(messy)
```

```
# A tibble: 1 x 2  
  first_name `2nd_col`  
    <dbl>    <dbl>  
1         1         2
```

```
commonnames(mtcars, iris)
```

```
character(0)
```

`commonnames()` is a convenience alias for the lower-level `common_names()`:

```
common_names(mtcars, iris)
```

```
character(0)
```

8.8 Filling in missing values within groups: `filldown()`, `fillup()`, `fillboth()`

`filldown()` carries the last non-missing value forward within each group (last observation carried forward); `fillup()` carries the next non-missing value backward; `fillboth()` does both, filling any remaining gaps from either direction.

```
visits <- data.frame(  
  id = c(1, 1, 1, 2, 2),  
  visit = c(1, 2, 3, 1, 2),  
  treatment = c("A", NA, NA, NA, "B")  
)  
filldown(visits, cols = "treatment", by = "id")
```

```
# A tibble: 5 x 3  
      id visit treatment  
  <dbl> <dbl> <chr>  
1     1     1      A  
2     1     2      A  
3     1     3      A  
4     2     1 <NA>  
5     2     2      B
```

```
fillup(visits, cols = "treatment", by = "id")
```

```
# A tibble: 5 x 3  
      id visit treatment  
  <dbl> <dbl> <chr>  
1     1     1      A  
2     1     2 <NA>  
3     1     3 <NA>  
4     2     1      B  
5     2     2      B
```

```
fillboth(visits, cols = "treatment", by = "id")
```

```
# A tibble: 5 x 3
      id visit treatment
  <dbl> <dbl> <chr>
1     1     1 A
2     1     2 A
3     1     3 A
4     2     1 B
5     2     2 B
```

9 Row and column utilities

9.1 Column metadata: colnames(), rownames(), classes(), uniques(), cardinality(), constants(), emptycols()

```
colnames(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

```
classes(iris)
```

```
# A tibble: 5 x 2
  column      class
  <chr>      <chr>
1 Sepal.Length numeric
2 Sepal.Width  numeric
3 Petal.Length numeric
4 Petal.Width  numeric
5 Species      factor
```

```
uniques(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
35	23	43	22	3

```
cardinality(iris, cols = "Species")
```

```
Species  
      3
```

```
constants(data.frame(a = 1, b = 1:2))
```

```
[1] "a"
```

```
emptycols(data.frame(a = c(NA, NA), b = 1:2))
```

```
[1] "a"
```

9.2 Finding blank rows: emptyrows()

```
emptyrows(data.frame(a = c(NA, "x", ""), b = c(NA, "y", NA)))
```

```
      a      b  
1 <NA> <NA>  
3      <NA>
```

9.3 Reordering columns: move(), firstcols(), lastcols()

```
move(mtcars, "hp", before = "mpg") |> colnames()
```

```
[1] "hp"    "mpg"   "cyl"   "disp"  "drat"  "wt"    "qsec"  "vs"    "am"    "gear"  "carb"
```

```
firstcols(mtcars, "wt") |> colnames()
```

```
[1] "wt"    "mpg"   "cyl"   "disp"  "hp"    "drat"  "qsec"  "vs"    "am"    "gear"  "carb"
```

```
lastcols(mtcars, "mpg") |> colnames()
```

```
[1] "cyl"   "disp"  "hp"    "drat"  "wt"    "qsec"  "vs"    "am"    "gear"  "carb"  "mpg"
```

9.4 Row subsets: firstrows(), lastrows(), samplerows(), samplefrac(), slice(), reverse()

```
firstrows(mtcars, 3)
```

```
# A tibble: 3 x 11
  mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  21     6   160   110  3.9   2.62  16.5    0    1    4     4
2  21     6   160   110  3.9   2.88  17.0    0    1    4     4
3 22.8    4   108    93  3.85   2.32  18.6    1    1    4     1
```

```
lastrows(mtcars, 2)
```

```
# A tibble: 2 x 11
  mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  15     8   301   335  3.54  3.57  14.6    0    1    5     8
2 21.4    4   121   109  4.11  2.78  18.6    1    1    4     2
```

```
set.seed(1)
samplerows(mtcars, 2)
```

```
# A tibble: 2 x 11
  mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 19.2    8   400   175  3.08  3.84  17.0    0    0    3     2
2 21.4    6   258   110  3.08  3.22  19.4    1    0    3     1
```

```
slice(mtcars, c(1, 3, 5))
```

```
# A tibble: 3 x 11
  mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  21     6   160   110  3.9   2.62  16.5    0    1    4     4
2 22.8    4   108    93  3.85   2.32  18.6    1    1    4     1
3 18.7    8   360   175  3.15   3.44  17.0    0    0    3     2
```

9.5 Sorting rows: `orderrows()`

```
orderrows(mtcars, by = "mpg", decreasing = TRUE) |> headtail(2)
```

```
# A tibble: 4 x 11
  mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  33.9     4  71.1    65  4.22  1.84  19.9     1   1     4     1
2  32.4     4  78.7    66  4.08  2.2   19.5     1   1     4     1
3  10.4     8 472     205  2.93  5.25  18.0     0   0     3     4
4  10.4     8 460     215   3     5.42  17.8     0   0     3     4
```

9.6 Column reordering: `relocate()`

```
relocate(mtcars, "hp", .before = "mpg") |> colnames()
```

```
[1] "hp"    "mpg"   "cyl"   "disp"  "drat"  "wt"    "qsec"  "vs"    "am"    "gear"  "carb"
```

9.7 Row position helpers: `rownumber()`

```
rownumber(c("a", "b", "c"))
```

```
[1] 1 2 3
```

9.8 Row-wise reductions across columns: `rowmin()`, `rowmax()`, `rowany()`, `rowall()`, `rowcount()`, `rowfirst()`, `rowlast()`, `rowapply()`

```
d <- data.frame(x = c(1, NA, 3), y = c(4, 5, NA), z = c(TRUE, FALSE, TRUE))
rowmin(d, cols = c("x", "y"), na.rm = TRUE)
```

```
[1] 1 5 3
```

```
rowmax(d, cols = c("x", "y"), na.rm = TRUE)
```

```
[1] 4 5 3
```

```
rowfirst(d, cols = c("x", "y"), na.rm = TRUE)
```

```
[1] 1 5 3
```

```
rowlast(d, cols = c("x", "y"), na.rm = TRUE)
```

```
[1] 4 5 3
```

```
rowapply(d, cols = c("x", "y"), fun = function(row) sum(row, na.rm = TRUE))
```

```
[1] 5 5 3
```

9.9 Applying a function to selected columns: `applycols()`, `convertcols()`

```
applycols(mtcars, cols = "mpg", fun = round) |> headtail(2)
```

```
# A tibble: 4 x 11
  mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    21     6   160   110   3.9   2.62  16.5     0     1     4     4
2    21     6   160   110   3.9   2.88  17.0     0     1     4     4
3    15     8   301   335   3.54  3.57  14.6     0     1     5     8
4    21     4   121   109   4.11  2.78  18.6     1     1     4     2
```

```
convertcols(mtcars, cols = "cyl", fun = as.character) |> types()
```

```
# A tibble: 11 x 3
  column class      typeof
  <chr>   <chr>      <chr>
1 mpg    numeric    double
2 cyl    character  character
3 disp    numeric    double
```

```
4 hp      numeric  double
5 drat     numeric  double
6 wt       numeric  double
7 qsec     numeric  double
8 vs       numeric  double
9 am       numeric  double
10 gear    numeric  double
11 carb    numeric  double
```

10 String manipulation

10.1 Whitespace and case: `trim()`, `squish()`, `lower()`, `upper()`, `titlecase()`, `sentencecase()`, `textlen()`

```
trim("  hello  ")
```

```
[1] "hello"
```

```
squish("  too  many  spaces  ")
```

```
[1] "too many spaces"
```

```
lower("HELLO"); upper("hello")
```

```
[1] "hello"
```

```
[1] "HELLO"
```

```
titlecase("hello world"); sentencecase("HELLO WORLD")
```

```
[1] "Hello World"
```

```
[1] "Hello world"
```

```
textlen(c("abc", NA))
```

```
[1] 3 NA
```

10.2 Substrings and padding: left(), right(), middle(), truncate(), padleft(), padright(), padcenter()

```
left("hello", 3); right("hello", 3); middle("hello", 2, 4)
```

```
[1] "hel"
```

```
[1] "llo"
```

```
[1] "ell"
```

```
truncate("a long sentence", 10)
```

```
[1] "a long ..."
```

```
padleft("7", 3, pad = "0"); padright("7", 3, pad = "0"); padcenter("hi", 6, pad = "**")
```

```
[1] "007"
```

```
[1] "700"
```

```
[1] "**hi**"
```

10.3 Pattern matching: contains(), matches(), startswith(), endswith(), countmatch(), locate(), locateall()

```
contains(c("apple", "banana"), "an")
```

```
[1] FALSE TRUE
```

```
matches(c("cat", "cats"), "cat")
```

```
[1] TRUE FALSE
```

```
startswith(c("apple", "banana"), "a")
```

```
[1] TRUE FALSE
```

```
endswith(c("apple", "banana"), "a")
```

```
[1] FALSE TRUE
```

```
countmatch("banana", "a")
```

```
[1] 3
```

```
locate("banana", "an")
```

```
[1] 2
```

```
attr(,"match.length")
```

```
[1] 2
```

```
attr(,"index.type")
```

```
[1] "chars"
```

```
attr(,"useBytes")
```

```
[1] TRUE
```

10.4 Extraction: `extract()`, `extractall()`, `extractnum()`, `extractint()`, `extractbetween()`

```
extract("order #4231", "[0-9]+")
```

```
[1] "4231"
```

```
extractall("a1b2c3", "[0-9]")
```

```
[[1]]
```

```
[1] "1" "2" "3"
```

```
extractnum("total: -12.5 kg")
```

```
[1] -12.5
```

```
extractbetween("[important]", "\\[", "\\]")
```

```
[1] "important"
```

10.5 Replace and remove: `replacetext()`, `removetext()`, `removeall()`, `replaceall()`

```
replacetext("a-b-c", "-", "_")
```

```
[1] "a_b_c"
```

```
removetext("a-b-c", "-")
```

```
[1] "abc"
```

```
replaceall(c("a", "b"), old = "a", new = "A")
```

```
[1] "A" "b"
```

10.6 Split and join: `splittext()`, `splitfirst()`, `splitlast()`, `jointext()`, `collapsetext()`

```
splittext("a-b-c", "-")
```

```
[[1]]
```

```
[1] "a" "b" "c"
```

```
splitfirst("a-b-c", "-"); splitlast("a-b-c", "-")
```

```
[1] "a"
```

```
[1] "c"
```

```
jointext("a", "b", "c")
```

```
[1] "abc"
```

```
collapsetext(c("a", "b", "c"), sep = ", ")
```

```
[1] "a, b, c"
```

10.7 Blank and encoding helpers: `isblank()`, `removeaccents()`, `normalizeunicode()`, `normalizeencoding()`, `transliterate()`

```
isblank(c("", " ", NA, "x"))
```

```
[1] TRUE TRUE TRUE FALSE
```

```
removeaccents("café")
```

```
[1] "cafe"
```

10.8 String distance and similarity: `textdist()`, `nearesttext()`, `similartext()`

```
textdist("cat", c("cat", "bat", "dog"))
```

```
      [,1] [,2] [,3]  
[1,]    0    1    3
```

```
nearesttext("kat", c("cat", "dog", "bird"))
```

```
[1] "cat"
```

10.9 Classification predicates: `isalpha()`, `isalphanumeric()`, `isnumerictext()`, `isintegertext()`, `isemail()`, `isurl()`

```
isalpha(c("abc", "a1c"))
```

```
[1] TRUE FALSE
```

```
isnumerictext(c("12.5", "abc"))
```

```
[1] TRUE FALSE
```

```
isemail(c("a@b.com", "not-an-email"))
```

```
[1] TRUE FALSE
```

```
isurl(c("https://example.com", "example.com"))
```

```
[1] TRUE FALSE
```

10.10 Recoding and grouping values: `recode()`, `collapsevalues()`, `collapselevels()`, `lump()`, `reorderlevels()`, `expandlevels()`

```
recode(c("a", "b", "c"), old = "a", new = "A")
```

```
[1] "A" "b" "c"
```

```
collapsevalues(c("cat", "dog", "bird"), groups = list(pet = c("cat", "dog")))
```

```
[1] "pet" "pet" "bird"
```

```
x <- rep(c("a", "b", "c", "d"), c(10, 5, 3, 1))  
lump(x, n = 2, other = "Other") |> table()
```

a	b	Other
10	5	4

```
f <- factor(c("low", "high", "mid"), levels = c("low", "mid", "high"))
reorderlevels(f, by = c("high", "mid", "low"))
```

```
[1] low  high mid
Levels: high mid low
```

```
expandlevels(f, "extra")
```

```
[1] low  high mid
Levels: low mid high extra
```

11 Parsing text into typed values

`parse*()` functions convert messy text into typed vectors, with an `na=` for sentinel missing-value strings and a `strict=` to error instead of silently returning NA on failure.

```
parseint(c("1", "2", "NA"))
```

```
[1] 1 2 NA
```

```
parsenum("1.234,56", decimal = ",", grouping = ".")
```

```
[1] 1235
```

```
parselogical(c("TRUE", "false", "T", "0"))
```

```
[1] TRUE FALSE TRUE FALSE
```

```
parsedate(c("2024-01-15", "15/01/2024"), formats = c("%Y-%m-%d", "%d/%m/%Y"))
```

```
[1] "2024-01-15" "2024-01-15"
```

```
parsedatetime("2024-01-15 08:30:00", formats = "%Y-%m-%d %H:%M:%S")
```

```
[1] "2024-01-15 08:30:00 UTC"
```

```
parsepercent("42%")
```

```
[1] 0.42
```

```
parsecurrency("$1,234.50")
```

```
[1] 1234
```

```
parsefailures(c("1", "x", "3"), parseint)
```

```
# A tibble: 1 x 2  
  index value  
  <int> <chr>  
1     2 x
```

12 Dates and times

12.1 Extracting components

```
d <- as.Date("2024-03-15")  
year(d); month(d); day(d); weekday(d); yearday(d); week(d); quarter(d)
```

```
[1] 2024
```

```
[1] 3
```

```
[1] 15
```

```
[1] "Friday"
```

```
[1] 75
```

```
[1] 11
```

```
[1] 1
```

```
t <- as.POSIXct("2024-03-15 08:30:45", tz = "UTC")
hour(t); minute(t); second(t)
```

```
[1] 8
```

```
[1] 30
```

```
[1] 45
```

12.2 Arithmetic and sequences: `adddays()`, `addweeks()`, `addmonths()`, `addyears()`, `datediff()`, `dateseq()`, `between dates()`

`addmonths()/addyears()` handle end-of-month overflow explicitly via `invalid = c("previous", "next", "missing", "error")` — e.g. what should January 31st plus one month become, given February doesn't have 31 days?

```
adddays(d, 10); addweeks(d, 2)
```

```
[1] "2024-03-25"
```

```
[1] "2024-03-29"
```

```
addmonths(as.Date("2024-01-31"), 1, invalid = "previous")
```

```
[1] "2024-02-29"
```

```
addmonths(as.Date("2024-01-31"), 1, invalid = "next")
```

```
[1] "2024-03-02"
```

```
datediff(as.Date("2024-01-10"), as.Date("2024-01-01"), units = "days")
```

```
[1] 9
```

```
dateseq(as.Date("2024-01-01"), as.Date("2024-01-05"))
```

```
[1] "2024-01-01" "2024-01-02" "2024-01-03" "2024-01-04" "2024-01-05"
```

```
between dates(d, "2024-01-01", "2024-12-31")
```

```
[1] TRUE
```

12.3 Rounding dates: floordate(), ceilingdate(), rounddate()

```
floordate(d, "month"); ceilingdate(d, "month"); rounddate(d, "month")
```

```
[1] "2024-03-01"
```

```
[1] "2024-04-01"
```

```
[1] "2024-03-01"
```

```
floordate(d, "week")
```

```
[1] "2024-03-11"
```

13 Numeric transforms and rolling windows

13.1 Rescaling and standardizing: rescale(), standardize(), center(), winsorize()

```
x <- c(1, 2, 3, 4, 100)
rescale(x); standardize(x); center(x)
```

```
[1] 0.0000 0.0101 0.0202 0.0303 1.0000
```

```
[1] -0.4815 -0.4585 -0.4356 -0.4127 1.7883
```

```
[1] -21 -20 -19 -18 78
```

```
winsorize(x, probs = c(0.1, 0.9))
```

```
[1] 1.4 2.0 3.0 4.0 61.6
```

13.2 Binning: quantilegroup()

```
quantilegroup(1:100, n = 4) |> table()
```

```
 [1,25.8] (25.8,50.5] (50.5,75.2] (75.2,100]  
         25         25         25         25
```

13.3 Period-over-period change and rank: percentchange(), percentrank(), denserank()

`denserank()` ranks values by sorted order with no gaps between ranks, tied values sharing a rank (the same semantics as SQL's `DENSE_RANK()`).

```
percentchange(c(100, 110, 99))
```

```
[1] NA 10 -10
```

```
percentrank(c(10, 20, 20, 30))
```

```
[1] 0.250 0.625 0.625 1.000
```

```
denserank(c(30, 10, 20, 10, 30))
```

```
[1] 3 1 2 1 3
```

13.4 Cumulative helpers: cumcount(), cumedist(), cummean()

```
cumcount(c("a", "b", "c"))
```

```
[1] 1 2 3
```

```
cumedist(c("a", "a", "b"))
```

```
[1] 1.0000 0.5000 0.6667
```

```
cummean(c(1, 2, 3))
```

```
[1] 1.0 1.5 2.0
```

13.5 Lag/lead and differencing: `difference()`, `lagvalue()`, `leadvalue()`

```
difference(c(1, 3, 6, 10))
```

```
[1] NA 2 3 4
```

```
lagvalue(1:5, n = 1)
```

```
[1] NA 1 2 3 4
```

```
leadvalue(1:5, n = 1)
```

```
[1] 2 3 4 5 NA
```

13.6 Rolling window functions: `rollmean()`, `rollsum()`, `rollmin()`, `rollmax()`, `rollmedian()`, `rollsd()`, `rollvar()`, `rollprod()`, `rollapply()`

All roll functions share the same interface: a `width`, an `align` (“right”/“left”/“center”), a `fill` for incomplete windows, `partial` to allow incomplete boundary windows instead of `fill`, and (where numerically meaningful) `na.rm`.

```
v <- c(1, 2, NA, 4, 5)
rollmean(v, width = 3, na.rm = TRUE)
```

```
[1] NA NA 1.5 3.0 4.5
```

```
rollsum(v, width = 3, na.rm = TRUE, partial = TRUE)
```

```
[1] 1 3 3 6 9
```

```
rollmax(v, width = 2, na.rm = TRUE)
```

```
[1] NA 2 2 4 5
```

```
rollapply(1:5, width = 3, FUN = sum, partial = TRUE)
```

```
[1] 1 3 6 9 12
```

14 Functional helpers

`map()/walk()/reduce()` are lightweight `functionals`-package-backed helpers for working with plain vectors and lists (not tables) — useful for the occasional loop-replacement inside a larger `basetable` pipeline.

```
map(1:3, function(x) x + 0.5)
```

```
[[1]]  
[1] 1.5
```

```
[[2]]  
[1] 2.5
```

```
[[3]]  
[1] 3.5
```

```
map_dbl(1:3, function(x) x + 0.5)
```

```
[1] 1.5 2.5 3.5
```

```
map_int(1:3, function(x) x * 2L)
```

```
[1] 2 4 6
```

```
map_chr(1:3, as.character)
```

```
[1] "1" "2" "3"
```

```
map_lgl(1:3, function(x) x > 1)
```

```
[1] FALSE TRUE TRUE
```

```
walk(c("a", "b"), function(x) invisible(x))  
reduce(1:4, `+`)
```

```
[1] 10
```

15 Closing notes

This document covers all of `basetable`'s exported functions. For the performance story (how the wrapper compares to raw `data.table`, and where its overhead was tracked down and fixed), see the companion `benchmarking` vignette.